Deep Transfer Learning for Intrusion Detection in Edge Computing Scenarios

Zeeshan Ali, *Graduate Student Member, IEEE*, Walter Tiberti, *Member, IEEE*, Andrea Marotta, *Member, IEEE*, and Dajana Cassioli, *Senior Member, IEEE*

***Abstract*—The rapidly evolving landscape of cyber threats poses significant challenges to network security, particularly in decentralized environments, such as *edge computing*. This article proposes an enhanced intrusion detection system (IDS) architecture that integrates transfer learning (TL) to create a unified *supermodel*, enabling adaptability and scalability for detecting emerging threats across diverse datasets (DS). The key contributions of this study include: 1) leveraging bidirectional encoder representations from transformer (BERT)-based feature extraction to enhance the semantic understanding of intrusion patterns; 2) employing an multilayer perceptron (MLP) classifier refined through TL to improve classification performance; 3) addressing class imbalance using synthetic minority over-sampling technique (SMOTE); and 4) optimizing model deployment by distributing the heaviest computational tasks for the creation of the unified *supermodel* across the edge nodes with the available capacity, thereby reducing latency and enabling accurate real-time threat detection by resource constrained IoT devices. The proposed TL-enabled *supermodel* is periodically updated and shared with the IoT devices, ensuring robust and adaptive security mechanisms without the need for extensive local training. Our experimental evaluation on CIC-IDS 2017 and NSL-KDD 2009 DS demonstrates the effectiveness of the approach, achieving 99% accuracy, precision, recall, and F1-score. Our results highlight the scalability, efficiency, and real-world applicability of our IDS framework, reinforcing its role in fortifying network security within highly dynamic cyber threat landscapes.**

***Index Terms*—Bidirectional encoder representations from transformers (BERTs), cybersecurity, edge computing, imbalanced dataset, intrusion detection system (IDS), multilayer perceptron (MLP), transfer learning (TL).**

I. Introduction

THE INTRODUCTION of edge computing marks a significant departure from traditional cloud-based architectures, offering a more responsive and efficient approach to data processing and analysis. This paradigm shift is driven by the need for real-time decision-making and the proliferation of devices generating vast amounts of data at the network's periphery. Edge computing capitalizes on the strategic placement of computing resources at the network's edge, fostering a distributed model well-suited for applications requiring instantaneous responses. In the context of IoT, smart cities, and autonomous systems, the integration of edge computing enhances the overall system performance by enabling data processing at the source. This approach reduces latency, optimizes bandwidth usage, and saves energy by minimizing the transmission of large volumes of raw data to centralized servers [1]. As a consequence, edge computing is particularly beneficial for scenarios where swift and localized data analysis is crucial, such as real-time monitoring of sensor data or rapid decision-making in autonomous vehicles.

The dynamic nature of edge environments, characterized by many interconnected devices and varying network conditions, underscores the importance of robust security measures. Intrusion detection becomes paramount in ensuring the integrity of data flows and the confidentiality of sensitive information within these distributed ecosystems. As edge computing continues its ascendancy, striking a balance between performance optimization and security becomes a central concern for architects and developers. Intrusion detection systems (IDSs) have emerged as critical components in ensuring the integrity and availability of computer systems by detecting and responding to unauthorized activities. However, the specific challenges inherent in decentralized environments, such as limited computational resources, dynamic network conditions, and the diversity of connected devices [2], underscore the crucial need for innovative approaches specifically tailored to enhance the effectiveness of intrusion detection mechanisms in these scenarios.

Machine learning (ML)-based IDS leverages the power of advanced algorithms and computational intelligence to enhance security postures. Unlike rule-based systems, ML-based IDS can learn and adapt from large datasets (DS), making them well-suited to identify novel and previously unseen threats. These systems employ various ML techniques, such as supervised, unsupervised, and deep learning (DL), to analyze network traffic, system logs, and other relevant data sources. By doing so, they can autonomously detect unusual patterns, anomalies, or suspicious behaviors that might indicate a cyber intrusion.

Transfer learning (TL) is a transformative approach in DL that focuses on reusing knowledge acquired from one domain or task to improve performance on a different but

Received 9 July 2025; accepted 4 August 2025. Date of publication 11 August 2025; date of current version 24 October 2025. This work was supported in part by the European Union—NextGenerationEU through the Italia Ministry of University and Research National Innovation Ecosystem under Grant ECS00000041 and through VITALITY under Grant CUP E13C22001060006; in part by the Partnership on "Telecommunications of the Future" under Grant PE00000001 and through the Program "RESTART" under Grant CUP E83C22004640001; and in part by the Project ISP5G+—CUP under Grant D33C22001300002, which is part of the SERICS Program under Grant PE00000014. *(Corresponding author: Zeeshan Ali.)*

The authors are with the Department of Information Engineering, Computer Science, and Mathematics, University of L'Aquila, 67100 L'Aquila, Italy (e-mail: zeeshan.ali@graduate.univaq.it; walter.tiberti@univaq.it; andrea.marotta@univaq.it; dajana.cassioli@univaq.it).

Digital Object Identifier 10.1109/JIOT.2025.3597892

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

related task. Instead of training a model from scratch, TL leverages pretrained models, which have already learned complex patterns from large DS, and adapts them to new tasks through fine-tuning. This approach accelerates the training process and reduces reliance on extensive labeled data, making it highly effective in scenarios with limited resources or data availability. TL has become a cornerstone in natural language processing (NLP), enabling highly accurate models with significantly reduced computational costs.

Recent advancements in large-language models (LLMs), such as bidirectional encoder representations from transformers (BERTs), have revolutionized NLP by enabling deep contextual understanding of complex data patterns. BERT is a powerful NLP model developed by Google in 2018 that has achieved state-of-the-art performance on various NLP tasks, such as text classification, question-answering, and capturing contextual relationships. Leveraging this capability, our work introduces a novel application for intrusion detection in edge computing environments. Although BERT and similar models have been used primarily in textual and semantic tasks, our approach is among the first to harness their powerful feature extraction capabilities to enhance network intrusion detection. By applying BERT to structured network traffic data, we uncover nuanced behavioral patterns that traditional models may overlook. This innovative integration not only boosts detection performance but also brings the strength of advanced DL to resource-constrained systems, marking a significant step forward in intelligent, real-time threat mitigation.

Network threats often exhibit sophisticated and evolving behaviors; thus, IDS should be able to capture intricate, nonlinear patterns in network flow data [3]. This is made possible by a particular artificial neural network (ANN) architecture that employs the nonlinear activation functions, i.e., the multilayer perceptron (MLP). Compared to linear classifiers, MLP exhibits a remarkable ability to manage high-dimensional data, a crucial advantage in network traffic analysis.

Edge computing networks, which involve IoT networks of resource-constrained devices, cannot rely on computationally intensive models. At the same time, highly dynamic and heterogeneous environments demand scalable and adaptable solutions to address evolving cyber threats. Additionally, the distributed nature of data collection at edge nodes results in diverse and nonuniform DS, complicating the task of creating a unified and comprehensive training model. Ensuring the generalizability of IDS models across varying DS and intrusion scenarios remains a critical challenge.

The above challenges highlight the pressing need for innovative approaches to develop scalable, adaptable, and resource-efficient IDS tailored to edge computing environments.

Our study evaluates a pioneering application of TL within IDS tailored for edge computing environments. In edge computing, where diverse devices collaborate and share data, our approach capitalizes on this distributed information to create a robust, adaptive intrusion detection model. The novelty of our approach lies in aggregating data from multiple devices to construct a comprehensive and diverse dataset. This combined dataset trains a unified *supermodel* synthesizing insights across the different sources. Once trained, the *supermodel* is deployed to IoT devices, continuously providing them with updated data for further model's refinement and adaptation over time.

An alternative, valuable approach to enhance adaptability is federated learning (FL). FL allows multiple edge nodes to collaboratively train models without sharing raw data, thereby improving privacy and scalability. Potential benefits of FL in IDS include privacy-preserving adaptability, reduced communication overhead, and enhanced global threat awareness. However, FL underlies challenges, such as aggregation complexity and adversarial security risk. Instead, our proposed TL approach offers a unique advantage in enhancing the precision and effectiveness of intrusion detection by capitalizing on the collective knowledge of diverse devices to identify and respond to novel threats in real-time, while optimizing resource utilization.

The contributions of this article can be summarized as follows.

1) We extend the effectiveness of BERT-based feature extraction with synthetic minority over-sampling technique (SMOTE) to handle DS' class imbalance to improve detection accuracy.
2) We compare the performance achieved by the proposed IDS using the most widely used variants of BERT architecture, namely, BERT-Base and BERT-Large. BERT-Base, with a vector size of 768, offers a lightweight and faster alternative suitable for resource-constrained environments, whereas BERT-Large, with a vector size of 1024, captures more complex representations, potentially enhancing detection performance.
3) We introduce a novel TL—enabled IDS architecture to create a unified *supermodel* that leverages knowledge acquired from diverse edge nodes to recognize transferable patterns across various intrusion scenarios, enhancing the adaptability of the IDS to novel and evolving threats. The *supermodel* is highly compact and lightweight, with a size of only 3.10 MB, particularly suitable for deployment on resource-constrained IoT devices. This efficiency ensures real-time intrusion detection with minimal computational overhead, facilitating practical deployment in distributed environments.
4) We enhance the downstream classification performance of network traffic data, by applying an MLP, aligning with the high scalability and low latency requirements of edge computing.
5) We validate our TL–based IDS model using two widely adopted IDS DS, CIC-IDS 2017, and NSL-KDD 2009, which, although not edge-specific, encompass a wide range of attack types and network behaviors. Our experimental results demonstrate significant gains in accuracy, precision, recall, and F1-score over state-of-the-art methods, highlighting the model's adaptability across diverse intrusion scenarios and its promising applicability to distributed settings.

The subsequent sections of the document are structured in the following manner. Section II presents a comprehensive discussion of the relevant and recently published literature.

Sections III and IV explain the proposed system model. The results are shown in Section V, which also offers a comprehensive discussion of our findings and their implications while outlining potential future research directions in this domain. Section VI serves as the concluding part of the paper.

II. Related Work

IDS can be viewed as a supervised anomaly detection task and phrased as a classification challenge. Recently, several research papers focused on extracting characteristic patterns from intrusion attacks to effectively detect them from typical network traffic. These approaches span various techniques, from conventional statistical to state-of-the-art DL-based approaches.

Feature selection is a preprocessing technique in ML aimed at reducing computational costs [4]. Its primary objective is to eliminate irrelevant features while simultaneously preserving or improving the performance of IDS.

To capture intricate features within network traffic data, a hybrid approach combining convolutional neural networks (CNNs) and long–short-term memory (LSTM) networks has been proposed by Dini et al. [3]. This method primarily focuses on centralized environments and is particularly effective in handling feature complexities; however, it is not flexible enough to generalize across diverse DS and to manage class imbalances in intrusion detection tasks. LSTM-based models have also been used for time series anomaly detection [5]. However, LSTM presents some limitations in capturing long-distance dependencies and has a stringent requirement for large amounts of labeled data.

To further enhance the efficiency of feature selection, Aslahi-Shahri et al. [6] employed a novel hybrid methodology that integrates a support vector machine (SVM) with a genetic algorithm (GA) to reduce the dimensionality of the dataset. Their experiment has shown how to distill the KDD CUP 99 dataset [7], of 41 features, into a more manageable subset of 10 essential features, and demonstrated the effectiveness of their innovative technique in increasing true positive rates while minimizing false positives. Although this technique is effective in streamlining feature sets and improving IDS performance, superior performance has been demonstrated by NLP methods aiming to directly improve feature representation, rather than relying solely on dimensionality reduction [8].

A powerful NLP method for feature extraction is BERT, which does not merely select existing features but generates new semantically rich feature representations from raw data [9]. By using BERT, the ability of models to understand complex patterns in the data is enhanced, which is especially crucial for the nuanced detection of intrusions.

Another effective approach for anomaly detection based on NLP has been introduced by Mimura and Tanaka [10]. This method leverages network interface packet analysis and primarily relies on source and destination IP information. The best-performing metric for this approach was an F1-score of 98%. However, decentralized networks, like, e.g., IoT, present unique challenges for intrusion detection, including the heterogeneity of devices, constrained computational and energy resources, and the vast volume of data generated in real-time.

In decentralized edge environments, traditional feature selection methods struggle with adaptability to evolving threats and scalability. Latif et al. [11] introduced a Deep TL-based IDS for heterogeneous industrial IoT (IIoT) networks, employing a tri-layer architecture that combines CNNs, GA and ensemble techniques. The framework achieved a 100% attack detection accuracy across 14 types of cyberattacks, demonstrating high effectiveness in IIoT environments. The integration of CNNs with GA and ensemble techniques offers robust detection capabilities and high accuracy. However, the use of CNNs and GAs might impose significant computational overhead, which is challenging in edge environments with limited resources.

The detection performance of ML-based IDS is strongly influenced by the *class imbalance* of DS. A novel model designed to address the challenge of class imbalance in supervised learning has been proposed by Zhang et al. [12]. Their approach combines the SMOTE with under-sampling for clustering using a Gaussian mixture model (GMM) to tackle the class imbalance problem effectively through resampling. The model introduced by Joloudari et al. [16] achieved 99.08% accuracy over 24 imbalanced DS by effectively handling imbalanced binary DS by CNN and SMOTE.

In scenarios where labeled data is scarce or expensive to obtain, a powerful approach is the *TL*, as it allows models to generalize knowledge acquired from different sources and synthesized in high-resource domains, ready to be shared with low-resource ones [17]. Applications of TL span a wide range of fields, including NLP, computer vision, and bioinformatics, where tasks, such as object recognition, language translation, and disease diagnosis have benefited greatly from this methodology. Unlike traditional learning approaches, which often require large amounts of task-specific data and computational resources, TL leverages pretrained models or features, significantly reducing the need for extensive data collection and training from scratch. Zong et al. [13] proposed an approach for intrusion detection that can transfer knowledge from a source domain with labeled data to a target domain with different statistical properties. In another study, Alrawashdeh and Purdy [14] employed TL in cyber threat hunting by leveraging labeled data from a source domain to enhance detection performance in a different but related target domain. Alhussien et al. [18] examined the impact of backdoor attacks on multisource TL models for intrusion detection. It finds that while backdoors in TL models are feasible, they have a lesser effect than on traditional ML models. Cheng et al. [19] introduced an efficient federated TL (FTL) framework for mobile-edge computing (MEC) intrusion detection, combining FL for privacy preservation and TL for enhanced training efficiency. Zhang et al. [15] proposed a privacy-preserving anomaly-based IDS for IIoT networks, employing FL and instance-based TL to address nonindependent and identically distributed (Non-IID) data challenges. A rank aggregation algorithm with weighted voting

is proposed, resulting in significant performance improvements, with 95.97% and 73.70% accuracy for AdaBoost and Random Forest. Yutao et al. [20] presented a visual geometry group (VGG)-RED approach for IoT intrusion detection, employing TL to migrate feature weights from the IoT intrusion detection DS. Results show a 2% accuracy improvement and 7%–13% reduction in time, demonstrating the model's efficacy in enhancing malicious attack detection in IoT environments while optimizing computational resources. Heterogeneous IoT environments are also targeted by Mehedi et al. [21], who proposed a deep TL-based IDS featuring effective attribute selection and a robust residual network (ResNet) model. The system outperforms existing methods, demonstrating enhanced accuracy, efficiency, and dependability. Saheed et al. [22] proposed the IoT-Defender framework, which combines a modified GA (MGA) for feature selection and a fine-tuned LSTM network within an edge computing environment for IoT intrusion detection. Evaluated on BoT-IoT [23], UNSW-NB15 [24], and N-BaIoT [25] DS using a Raspberry Pi, the model achieved 99.41% accuracy and 99.78% detection rate. Masum [26] proposed a network intrusion detection system (NIDS) classification framework using TL from the VGG-16 model, comparing its performance with VGG-19 [27], mobile network (MobileNet) [28], ResNet-50, and Inception-V3. By extracting features from ImageNet [29] pretrained weights and applying a deep neural network (DNN) for classification on the NSL-KDD 2009 dataset, the framework achieved an accuracy of 81.77% for binary and 78.39% for multiclass classification. Saheed and Misra [30] presented an explainable, privacy-preserving DNN framework for anomaly detection in cyber–physical systems (CPSs) enabled IoT networks, integrating shapley additive explanations (SHAPs) for interpretability. Tested on Edge-IIoTset [31] and X-IIoTID [32] DS, the model achieved near-perfect accuracy, demonstrating its effectiveness in intrusion detection while reducing false positives.

Compared to existing state-of-the-art IDS frameworks, including those in [22], [30], and [26], the originality of our proposed TL-enabled *supermodel* resides on the integration of BERT-based semantic feature extraction and a scalable edge-oriented deployment strategy. While prior works employ traditional deep models like ResNet, LSTM, and VGG for intrusion detection, our system introduces a unified architecture that leverages language-based embeddings to capture deeper contextual patterns of network intrusions.

The related work analysis has highlighted several valuable strategies for feature selection and class imbalance mitigation, often relying on conventional and DL techniques. In our previous study, we proposed an IDS framework leveraging SMOTE to handle class imbalance through oversampling and undersampling, a BERT transformer-based model for tokenization and feature extraction, and an MLP for classification, achieving robust results on individual DS [8]. We demonstrated that the BERT model excels in capturing nuanced contextual information and robust feature representations from network traffic data.

Building on this foundation, this article extends our prior work by introducing a TL-based paradigm for IDS tailored to the unique challenges of edge computing employed in IoT networks. The integration of TL to our framework enables the creation of a unified *supermodel* that generalizes knowledge across multiple DS, improving scalability and adaptability to evolving intrusion patterns across multiple DS. Unlike CNN-LSTM hybrids, our methodology prioritizes generalization and feature diversity, ensuring enhanced accuracy and resilience in detecting novel threats across different scenarios captured in the DS. This advancement ensures enhanced attack prediction capabilities while addressing the dynamic and decentralized nature of edge environments for IoT.

In summary, while several studies have explored SVM, GA, CNN, GMM and LSTM for feature extraction and intrusion classification, there remains a significant gap, as discussed in Table I. This study aims to bridge this gap by introducing a novel IDS architecture that integrates TL into a unified *supermodel*. The proposed work leverages the BERT-based transformer model for the feature extraction to improve the semantic understanding of intrusion patterns and employs a DL-based MLP classifier refined through TL to enhance classification performance. Hence, it offers a more comprehensive solution to the challenges identified in the literature.

III. System Model

The system model is shown in Fig. 1, along with the network architecture for the intelligent edge deployment of the proposed TL-based IDS in IoT scenarios.

The proposed architecture includes three layers.

1) The lower layer consists of multiple IoT networks, each serving a specific application domain, such as medical applications, Industry 5.0 and automotive; the IoT devices do not have sufficient resources to train their own ML model for intrusion detection and rely on the "Edge Layer" to offload this and other heavy computational tasks.
2) Lying on top of the IoT networks, the "Edge Layer" is made up of a multitude of powerful edge devices in charge of running heavy computational tasks on behalf of IoT devices belonging to the served IoT network; among these heavy tasks, edge nodes run the preprocessing of the DS acquired from the served IoT networks to train its own model, referred to as *standalone model (SAM)*. Every edge node is independent of the others in this training task, consisting of the same pipeline for all nodes to ensure consistency and compatibility of "local SAMs." The edge nodes are all connected through Internet and can apply the TL to improve their IDS model by acquiring the embeddings from different DS shared by other edge nodes and creating the *TL supermodel*. The details are provided in Section IV.
3) The top layer represents the "Data Acquisition Layer," responsible for collecting the relevant observation data and composing the DS to be fed to the "Edge Layer" for both the SAM and the *TL supermodel*.

TABLE I
COMPARISON OF EXISTING APPROACHES AND PROPOSED WORK

Author(s)	Approach	Addressed Challenge(s)	Limitation(s)	Proposed Work's Advantage
[3]	CNN-LSTM Hybrid for Feature Extraction	Feature extraction in centralized environments	Limited Generalizability	Our work generalizes across multiple datasets and enhances flexibility.
[6]	SVM with Genetic Algorithm for Feature Selection	Dimensionality reduction and feature selection	Does not leverage NLP techniques for feature representation	Our work employs BERT for richer, semantic feature extraction.
[10]	Packet Analysis for Intrusion Detection	Anomaly detection in network traffic	Focused on source and destination IPs, limited to network packet analysis	Our approach uses BERT to capture more complex, semantic intrusion patterns.
[11]	Deep TL-based IDS for Industrial IoT	High accuracy in IIoT networks	Computationally heavy for edge environments	Our approach optimizes computational tasks by distributing across edge nodes.
[12]	SMOTE with Under-sampling for Class Imbalance	Class imbalance in datasets	Might not address all forms of class imbalance effectively	We combine SMOTE with TL to address class imbalance more robustly and efficiently.
[13]	TL-based IDS for Knowledge Transfer Across Domains	Knowledge transfer for intrusion detection	Limited generalization across diverse datasets	Our unified *supermodel* adapts to multiple datasets and intrusion patterns.
[14]	Transfer Learning for Cyber Threat Hunting	Enhancing detection performance with limited target domain data	Focuses only on transferring knowledge from related domains	Our approach broadens the scope, creating a more adaptable *supermodel* across datasets.
[15]	Privacy-Preserving TL for IIoT	Privacy in Federated Learning with TL	Focuses on privacy but does not directly address performance issues	Our work balances with performance and enhancing detection capabilities.
Proposed	TL-based IDS with BERT Feature Extraction and MLP Classifier	Scalable, adaptable IDS for edge computing in IoT environments	Limited by previous methods' inability to handle evolving threats in decentralized environments	Enhanced scalability adaptability and feature richness through BERT and TL integration.

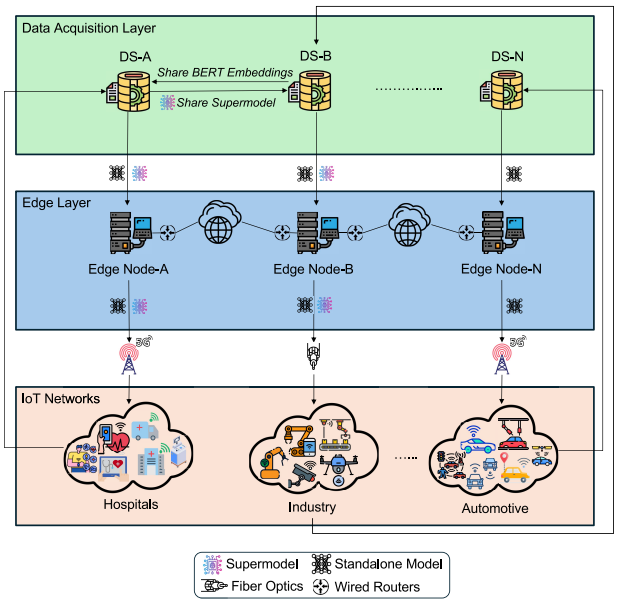


Fig. 1. System model and network architecture for intelligent edge deployment of the proposed TL-based IDS.

The novelty of our approach lies in the introduction of TL in the Edge Layer: whenever a trained model is available at one node, it is advertised among all other edge nodes, that may acquire such a trained model, greatly benefiting from this knowledge to extend its own model by creating a *supermodel* through TL. Hence, as explained in the following, our method leverages the knowledge gained from an initial dataset, i.e., the base model (BM), built on one edge node to enhance performance by acquiring new knowledge from another dataset available at a different edge node. Certainly, the presented methodology can be applied to any number of nodes participating in the TL process.

IV. TRANSFER LEARNING METHODOLOGY

The principle diagram depicted in Fig. 2 illustrates a collaborative edge-based TL framework involving two edge nodes, "edge node A (EN-A)" and "edge node B (EN-B)," designed to support both SAM training and TL-based *supermodel* for their IoT devices, as described in the following.

A. Standalone Model's Training

Every edge node is able to train its own SAM by applying the procedure summarized in Algorithm 1. This includes dataset preprocessing (Step Ⓟ) and feature extraction using Google BERT (Step Ⓖ), followed by the classification using an MLP model. Although these steps have been extensively described in [8], for the convenience of the reader, we summarize the main elements below.

With reference to the EN-B box in the lower part of Fig. 2, the data preprocessing pipeline (Step Ⓟ) performs tasks, such as SMOTE, duplication removal, and label mapping. Then, to ensure that the input representations capture contextual semantic information and to reduce the risk of overfitting to specific dataset characteristics, our methodology uses Google BERT. In the present work, we extend the analysis we proposed in [8] by testing two variants of BERT.

1) *BERT-Large-Uncased* consisting of 24 layers, 16 attention heads, and a hidden size of 1024.
2) *BERT-Base-Uncased* with its 12 layers, 12 attention heads, and a hidden size of 768.

Input sequences are tokenized with a maximum length of 128, applying truncation and padding. Feature extraction is performed using the CLS token representations from the last

Algorithm 1 Algorithm for the Creation of the SAM as in [8]

procedure MAIN(Dataset-A (DS-A))
  Input:
    DS-A
  Output:
    Base model **BM**: A classifier capable of making predictions on DS-A.
  Step 1: Load the DS-A and perform the *Dataset Preprocessing*
  Step 2: Tokenization and Feature Extraction using Google *BERT Base Transformer*
  Step 3: Build, Train and Test *MLP Classifier*:
    a. Training the MLP classifier on the BERT extracted train features of the DS-A.
    b. Base model **BM** is created and tested on the DS-A test features.
end procedure

hidden layer, processed in batches of 100 samples to optimize memory usage.

Our MLP model is configured with a single hidden layer containing 100 neurons, a maximum iteration limit of 200, and a fixed random state of 42 to ensure consistent results. Finally, the MLP classifier applies multiple regularization techniques to the extracted features. We incorporate dropout layers with a rate of 0.5 to randomly deactivate neurons during training, preventing the model from becoming overly reliant on specific patterns. Additionally, early stopping with a patience value of 10 is employed to monitor validation performance and halt training when overfitting is detected, ensuring that the model learns meaningful representations rather than memorizing the training data.

Furthermore, L2 regularization (also known as weight decay) is applied to the MLP classifier to help prevent overfitting. This technique works by adding a penalty term to the loss function that is proportional to the sum of the squared values of the model's weights. In simple terms, the model is discouraged from assigning large values to the weights during training, which helps reduce the complexity of the learned patterns. As shown in (1), the total loss includes both the original classification loss and the L2 penalty term. Here $L_{original}$ is the typical loss, w_i are the individual weights of the model, and α is a hyperparameter that controls how strongly the penalty is applied

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \alpha \sum_i w_i^2. \tag{1}$$

In our case, we set $\alpha = 0.0001$, which means the regularization is relatively mild but still sufficient to push the model toward smaller weights. This helps the model generalize better by reducing its sensitivity to noise or outliers in the training data. In effect, L2 regularization acts like a "weight smoother," promoting simpler models that are more likely to perform well on new, unseen data.

The SAM can be directly shared with the IoT network(s) connected to the edge node for intrusion detection or may be used as the BM for the creation of the *supermodel* as explained in Section IV-B.

Algorithm 2 Algorithm for TL From *BM-X* to *Supermodel SM-Y* for Two Edge Nodes **X** and **Y**

procedure MAIN(BM-X, Dataset-Y (DS-Y))
  Input:
    BM-X: Load the BM created at the edge node **X**
    DS-Y: Acquiring DS-Y at the edge node **Y**
  Output:
    supermodel **SM-Y**: A classifier capable of making predictions on both datasets **X** and **Y**.
  Step 1: Load the DS-Y and perform the *Dataset Preprocessing* Ⓟ
  Step 2: Tokenization and Feature Extraction using Google *BERT Base Transformer* Ⓖ
  Step 3: BM *Knowledge Transfer* Ⓚ using TL and fine-tuning:
    a. The BM knowledge from **BM-X** is transferred using the proposed TL approach.
    b. A new classification head Ⓗ is added to handle new classes in DS-Y, followed by fine-tuning.
    c. The *supermodel* **SM-Y**, enhanced via TL & fine-tuning Ⓣ, is created and tested Ⓢ.
end procedure

B. Supermodel's Creation

The procedure for the *supermodel*'s creation is summarized in Algorithm 2 and is shown in the upper box of the principle diagram of Fig. 2, the one related to EN-A. The *supermodel*'s creation relies on a BM, which is exactly the SAM generated at the edge node that hosts the TL, for instance the EN-A, applying the procedure described in Section IV-A. This SAM can be either used as a BM for the TL or deployed, through the transfer gate (TG) indicated in Fig. 2, to the IoT devices served by EN-A as an IDS based solely on the knowledge acquired by processing the DS-A. Let's assume, without any loss of generality, that EN-B aims to participate in the TL, thus it replies positively to the EN-A advertisement. EN-B has already trained its own SAM using DS-B and has stored the corresponding Google BERT embeddings obtained at Step Ⓖ. These Google BERT embeddings are then shared with EN-A via the receiving gate (RG) shown in Fig. 2 to create the *supermodel* and enhance the IDS performance through TL.

Once the TL engine in EN-A has acquired the BM-A from the BM gate and the Google BERT embeddings from the RG interfaced to EN-B, as indicated in Fig. 2, knowledge transfer is performed (Step Ⓚ), a new classification head is introduced (Step Ⓗ), and fine-tuning is applied (Step Ⓣ) to build the *supermodel* (Step Ⓢ) based on both DS-A and DS-B. Next, this *supermodel* is shared through the sending gate (SG) and received by EN-B via its RG. Now, the *supermodel* is ready to be deployed on EN-B's and EN-A's IoT devices through their own TG.

The challenge behind the TL approach is to load the pretrained *BM* on DS-B while preserving the knowledge

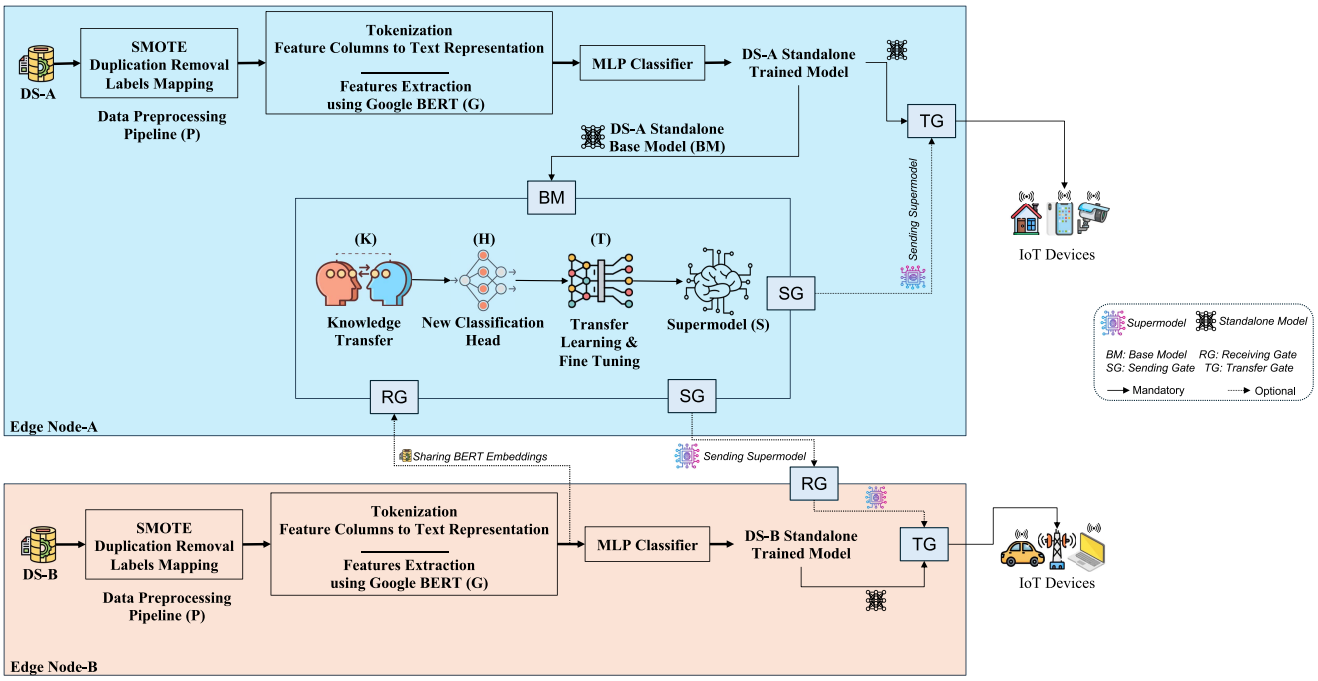


Fig. 2. Principle diagram of the proposed TL methodology for IDS in a decentralized IoT environment.

gained from its initial training without incurring the well-known "catastrophic forgetting" problem: when models are trained on new data, there is a risk they might overwrite or "forget" previously learned information [33]. To combat this issue, we employ a strategy designed on purpose, i.e., the "replay" technique, which operates by reintroducing a subset of DS-A during training on DS-B. This ensures the *supermodel* retains the knowledge from DS-A while learning from the new one.

Our practical implementation starts by unfreezing the *BM* top layers and loading the TensorFlow's StringLookup layer within the MLP, which is responsible for mapping string labels (i.e., categorical values) to integer encodings. Commonly integrated into preprocessing pipelines for ML models, the StringLookup layer offers a seamless approach to managing categorical data in a numerical format. By leveraging this layer, the model can effectively process and learn from categorical information, such as class labels, thereby improving compatibility with neural network architectures. We introduce the TensorFlow's StringLookup layer to cope with errors or misinterpretations in the model predictions due to "out-of-vocabulary (OOV)" tokens, i.e., string class labels not seen during the BM training phase. Actually, the StringLookup layer allows the specification of a unique OOV token, which is an integer representation assigned to any string not found in the DS-A. Essentially, all unseen labels are mapped to this OOV token. The DS-B features are loaded, combined with a subset (20%) of the DS-A, and then scaled. To enhance the *supermodel* efficiency, right after merging the OOV tokens from both DS, we update the classification head to effectively accommodate the consolidated class categories.

To avoid potential overfitting that can result from unfreezing the BM's top layers, we employ optimization techniques, such as *early stopping* and *learning rate scheduler* to enhance the generalizability of the model. Early stopping is a regularization method that monitors the model's performance on a validation dataset throughout training. If performance in the validation set deteriorates, training is halted early to prevent overfitting, ensuring that the *supermodel* maintains strong generalization capabilities on unseen data. *Learning rate scheduler* dynamically adjusts the step size during optimization. Initially, a higher learning rate accelerates convergence, and as the model approaches optimal parameters, the learning rate is decreased to fine-tune and avoid overfitting. Our experimental results (Section V) confirm that these strategies effectively mitigate overfitting, enabling the IDS to maintain high accuracy across multiple DS while demonstrating strong adaptability to real-world network traffic.

C. Computational Load and Scalability in Edge Deployments

The efficient deployment of IDSs in edge computing environments requires balancing model complexity with resource constraints. Our proposed TL-enabled IDS architecture addresses this challenge by offloading computationally intensive tasks to edge nodes, thereby reducing the overhead on resource-limited IoT end devices.

All edge nodes are able to independently train and deploy their own SAMs while optionally participating in a collaborative TL process. This approach fully benefits from the hierarchical and distributed architecture of edge computing where heavy training tasks are performed at the edge nodes and the sole *precomputed lightweight model*, be it the SAM or

TABLE II
EDGE NODES, DATASET CHARACTERISTICS, AND MODEL SIZE

Edge Nodes	Dataset Name	No of Records	Dataset Size	Trained Model Size
Edge Node A	CIC-IDS 2017 (DS-A)	2,574,984	843.60 MB	1.10 MB (Base Model)
Edge Node B	NSL-KDD 2009 (DS-B)	148,517	16.60 MB	3.10 MB (Super Model)

the *supermodel*, is transferred to IoT devices. These are only involved in the basic task of sharing relevant raw data, i.e., the DS-X, thus saving energy and resources.

This innovative approach has a twofold advantage. On one hand, it represents a novel means of *enhancing classification capabilities* by disseminating across IoT devices the knowledge gained in the central learning process, then it becomes particularly effective in distributed environments. On the other hand, the transfer of the *supermodel* is lightweight compared to the full dataset, thus it best matches the capabilities of resource-constrained devices in terms of both storage and computational capacity.

Additionally, the final *supermodel* consolidates *essential knowledge* from multiple edge nodes while discarding unnecessary parameters, optimizing storage efficiency.

As shown in Table II, the full dataset's size used to train the BM is about 1 GB, while the size of DS-B used to train the *supermodel* is 16.6 MB, which could be too large to be managed by the IoT devices. Instead, the BM size is very small, i.e., 1.1 MB, and the size of the *supermodel* shared with the N-end devices is about 3.10MB, which significantly reduces memory and computational demands in the deployment phase. Hence, the computational load is entirely at edge nodes, utilizing BERT-based feature extraction and an MLP classifier.

Concerning the scalability of the proposed approach, it is worth to highlight that the *supermodel* is periodically updated and shared across edge IoT devices, ensuring adaptability to evolving threats. Increasing the number of edge nodes allows parallel fine-tuning, enhancing detection capabilities without increasing central processing overhead. By leveraging TL, each new dataset improves the *supermodel* without requiring full retraining from scratch.

V. VALIDATION RESULTS

We validate the proposed methodology by running experiments on the distributed system architecture shown in Fig. 3, implemented on a high-performance computing platform optimized to handle large DS and complex tasks.

We utilized an NVIDIA A40 GPU shared across two virtual machines, designated as EN-A and EN-B. The BM training was conducted on EN-A and the trained model was shared with EN-B for TL and fine-tuning. This process resulted in a highly optimized *supermodel* which was then deployed to IoT devices for real-time intrusion detection, ensuring efficient and scalable threat mitigation in distributed environments. The source code for our TL-based supermodel is available on (https://github.com/drzeeshanaliofficial/Deep-Transfer-Learning-for-Intrusion-Detection-in-Edge-Computing-Scenarios), and the README contains detailed descriptions of the Python files.

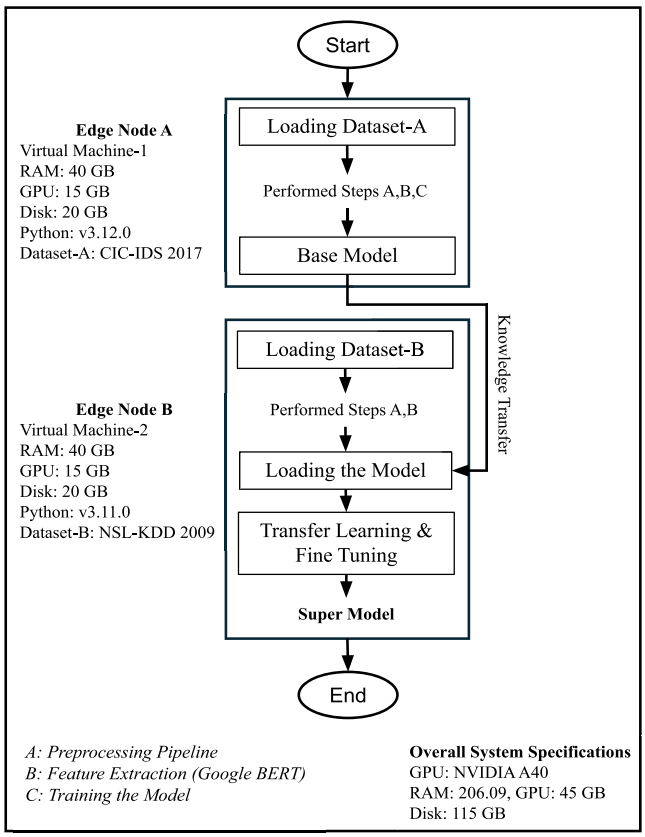


Fig. 3. Distributed system architecture used for the TL experiments.

A. Input Datasets

To emulate the heterogeneity of DS collected from different edge nodes, we adopt two DS widely used in IDS for edge computing in IoT networks [34], [35], [36]: the CIC-IDS 2017 [37] as the DS-A at the EN-A and the NSL-KDD 2009 [7] as the DS-B at the EN-B.

The CIC-IDS 2017 dataset, collected by the Canadian Institute for Cybersecurity, is widely used for NIDS since it contains a variety of network traffic data labeled as both normal and attack traffic, associated to the most common attack types, such as DoS, DDoS, port scanning, and more. Traffic data was captured in a controlled environment, simulating real-world scenarios, and includes detailed information, such as packet and flow-level features, resulting in the 79 distinct features shown in Table III, which form a rich foundation for our in-depth analysis.

The NSL-KDD dataset is a modified version of the original KDD Cup 1999 dataset, and contains a set of labeled network traffic data, with each instance representing a network connection characterized by a variety of features (up to 42), such as connection duration, protocol type, and error rates, listed in

TABLE III
FEATURES OF CIC-IDS 2017 DATASET

F#	Feature Name	F#	Feature Name	F#	Feature Name	F#	Features
1	Destination Port	21	Fwd IAT Total	41	Pkt Length Mean	61	Bwd Avg Pkt/Bulk
2	Flow Duration	22	Fwd IAT Mean	42	Pkt Length Std	62	Bwd Avg Bulk Rate
3	Total Fwd Pkt	23	Fwd IAT Std	43	Pkt Len Variance	63	Subflow Fwd Packs
4	Total Bwd Pkt	24	Fwd IAT Max	44	FIN Flag Count	64	Subflow Fwd Bytes
5	Total Len of Fwd Pkt	25	Fwd IAT Min	45	SYN Flag Count	65	Subflow Bwd Packs
6	Total Len of Bwd Pkt	26	Bwd IAT Total	46	RST Flag Count	66	Subflow Bwd Bytes
7	Fwd Pkt Len Max	27	Bwd IAT Mean	47	PSH Flag Count	67	Init Win bytes Fwd
8	Fwd Pkt Len Min	28	Bwd IAT Std	48	ACK Flag Count	68	Init Win bytes Bwd
9	Fwd Pkt Len Mean	29	Bwd IAT Max	49	URG Flag Count	69	Act Data Pkt Fwd
10	Fwd Pkt Len Std	30	Bwd IAT Min	50	CWE Flag Count	70	Min Seg Size Fwd
11	Bwd Pkt Len Max	31	Fwd PSH Flags	51	ECE Flag Count	71	Active Mean
12	Bwd Pkt Len Min	32	Bwd PSH Flags	52	Down/Up Ratio	72	Active Std
13	Bwd Pkt Len Mean	33	Fwd URG Flags	53	Average Pkt Size	73	Active Max
14	Bwd Pkt Len Std	34	Bwd URG Flags	54	Avg Fwd Seg Size	74	Active Min
15	Flow Bytes/s	35	Fwd Header Len	55	Avg Bwd Seg Size	75	Idle Mean
16	Flow Pkt/s	36	Bwd Header Len	56	Fwd Header Len. 1	76	Idle Std
17	Flow IAT Mean	37	Fwd Pkt/s	57	Fwd Avg Bytes/Bulk	77	Idle Max
18	Flow IAT Std	38	Bwd Pkt/s	58	Fwd Avg Pkt/Bulk	78	Idle Min
19	Flow IAT Max	39	Min Pkt Len	59	Fwd Avg Bulk Rate	79	Label

TABLE IV
FEATURES OF NSL-KDD 2009 DATASET

F#	Feature name	F#	Feature name	F#	Feature name
1	Duration	15	Su attempted	29	Same srv rate
2	Protocol type	16	Num root	30	Diff srv rate
3	Service	17	Num file creations	31	Srv diff host rate
4	Flag	18	Num shells	32	Dst host count
5	Src bytes	19	Num access files	33	Dst host srv count
6	Dest bytes	20	Num outbound cmds	34	Dst host same srv rate
7	Land	21	Is host login	35	Dst host diff srv rate
8	Wrong fragment	22	Is guest login	36	Dst host same src port rate
9	Urgent	23	Count	37	Dst host srv diff host rate
10	Hot	24	Srv count	38	Dst host error rate
11	Num failed logins	25	error rate	39	Dst host srv error rate
12	Logged in	26	Srv error rate	40	Dst host error rate
13	Num compromised	27	error rate	41	Dst host srv error rate
14	Root shell	28	Srv error rate	42	label

Table IV. The dataset includes both normal and attack traffic, with the attacks categorized into various types, such as DoS, R2L, U2R, and probing.

Both DS provide a comprehensive range of network flow and session-level features that closely align with the types of telemetry and behavior patterns typically observable in IoT devices and systems. For instance, features like flow duration, packet lengths, byte counts, interarrival times (IATs), and header statistics in CIC IDS 2017 are directly reflective of network traffic characteristics at the edge, where lightweight monitoring tools capture such parameters for real-time anomaly detection. Similarly, the NSL-KDD dataset includes key indicators, such as source/destination bytes, protocol type, service usage, and connection flags—features that are equally relevant in edge networks where constrained devices often interact over specific services and protocols. Moreover, both DS incorporate behavioral and statistical insights (e.g., login attempts, connection error rates, session flags) that are crucial for detecting intrusion or abnormal activity in distributed edge settings. The presence of such overlapping low-level and session-based features ensures that the analytical models trained on these DS can generalize well to edge scenarios, making their inclusion not only appropriate but essential for studying security in edge computing contexts.

The attacks covered by these DS are detailed in Tables V and VI, where DoS stands for Denial of Service, DDoS for Distributed DoS, R2L for Root to Local and U2R for User to Root attacks. These attack types are commonly encountered in distributed networks where IoT devices process data closer to the source, making it critical to detect and mitigate such threats in real-time to ensure the security and reliability of edge networks. Our choice of these DS is motivated by their widespread recognition within network security and intrusion detection, as well as their varied characteristics that collectively enable comprehensive model evaluation and validation. However, while they cover a wide range of attack types and normal traffic, we acknowledge that they may not capture all possible variations in network conditions that could be encountered in different edge environments, thus generating potential biases. To address this concern and to assess the model's performance and robustness in real-world scenarios, ensuring its generalizability across diverse edge environments,

TABLE V
CIC-IDS 2017 DATASET (REFERRED TO AS DS-A IN OUR SYSTEM MODEL)

No	Attacks	Attacks Found	Original Records	Augmented Records
1	BENIGN	BENIGN	2,096,484	Not Augmented
2	Brute Force	FTP-Patator SSH-Patator	9,152	12,000
3	DoS	DoS GoldenEye DoS Hulk DoS Slowhttptest DoS Slowloris Heartbleed Bot	195,712	205,000
4	Web Attacks	Web Attack - Brute Force, Web Attack - SQL Injection, Web Attack - XSS	2,143	10,000
5	Infiltration	Infiltration	36	1,500
6	Port Scan	Port Scan	90,819	100,000
7	DDoS	DDoS	128,016	150,000

TABLE VI
NSL-KDD 2009 DATASET (REFERRED TO AS DS-B IN OUR SYSTEM MODEL)

No	Labels	Training Set	Testing Set	Augmented Records
1	Normal	67,343	9,711	Not Augmented
2	DoS	45,927	7,460	Not Augmented
3	Probe	11,656	2,421	Not Augmented
4	R2L	995	2,885	8,000
5	U2R	52	67	5000

our future work will involve validating the model on additional DS collected from various edge nodes across different network environments.

B. Dataset Preprocessing

Both DS present a significant class imbalance, e.g., the BENIGN class in the CIC-IDS 2017 dataset (Table V) has over 2 million records, whereas classes, such as Web Attack and Infiltration have only 2143 and 36 records, respectively, and the NSL-KDD 2009 dataset (Table VI), demonstrates a pronounced imbalance in the R2L and U2R classes, with only 995 and 52 samples in the training set, respectively, compared to tens of thousands of samples in the "Normal" and "DoS" classes. Without addressing this class imbalance problem, the model would likely struggle to learn effective patterns for these minority classes, further reducing the efficacy of the IDS. To address this issue and improve the model's performance in detecting minority class intrusions, we augment the number of records for each attack class by using SMOTE [8], resulting in the "Augmented Records" listed in the last columns of Tables V and VI. This approach ensures that our IDS remains robust and effective, even when faced with highly imbalanced data distributions. While other techniques, such as ADASYN and cost-sensitive learning, are valid options, they often introduce additional complexity and require careful tuning of hyperparameters to achieve optimal results. Furthermore, methods like ADASYN can sometimes result in the oversynthesis of minority class instances, which may negatively impact the model's generalization ability. In contrast, SMOTE offers a more straightforward and well-established approach that balances performance enhancement with the preservation of data distribution.

As detailed in our prior work [8], we eliminate duplicate samples to ensure the dataset's integrity by preventing redundancy and bias, which could otherwise skew ML model outcomes. We then perform label mapping, i.e., the conversion of categorical labels into numerical values, to facilitate accurate processing and analysis by ML models. These preprocessing steps lay a strong foundation for developing robust and unbiased IDS frameworks.

As an upgrade of our previous work [8], we process the DS by using the most widely used BERT architectures, i.e., BERT-Base and BERT-Large, for feature extraction at both edge nodes A and B. BERT-Base consists of 12 transformer layers, 12 attention heads, and a hidden size of 768, making it computationally efficient while still capturing meaningful contextual representations. On the other hand, BERT-Large features 24 transformer layers, 16 attention heads, and a hidden size of 1024, enabling it to learn more complex patterns at the cost of increased computational demands.

Finally, to assess the ML model's efficacy, the preprocessed DS underwent stratified division into distinct training and testing subsets. Specifically, 80% of data was allocated for training purposes, while the remaining 20% was reserved for rigorous testing. This deliberate partitioning ensures a representative distribution of instances across both sets, maintaining a proportional representation of different classes in both the training and testing phases.

C. Evaluation Metrics

The evaluation is based on the five metrics Accuracy, Precision, Recall, F1-Score and Confusion Matrix, expressed as functions of the key evaluation terms defined in Table VII.

Accuracy represents the overall effectiveness of the classification process, and is calculated as

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \tag{2}$$

Precision is used to assess the accuracy of positive predictions in terms of the percentage of instances correctly

TABLE VII
DEFINITIONS OF TERMS: TP, TN, FP, FN

Term	Definition
True Positive (TP)	Correctly identified positive instances
True Negative (TN)	Correctly identified negative instances
False Positive (FP)	Instances wrongly identified as positive
False Negative (FN)	Instances wrongly identified as negative

TABLE VIII
CONFUSION MATRIX

Actual	Predicted	
	Positive	**Negative**
Positive	True Positives (TP)	False Negatives (FN)
Negative	False Positives (FP)	True Negatives (TN)

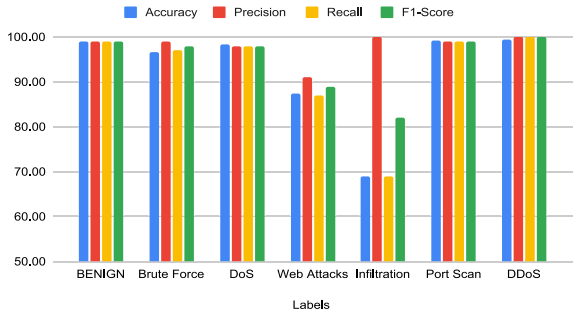


Fig. 4. Performance evaluation of BERT-base super model on the CIC-IDS 2017 dataset.

predicted by the model as positive (e.g., abnormal)

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{3}$$

Recall, also known as sensitivity or true positive rate, measures the ability of the model to identify all positive instances correctly

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{4}$$

Especially useful when dealing with imbalanced DS, where one class is more dominant than the others, the F1-Score provides a balanced measure of the model's performance between precision and recall

$$\text{F1-Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{5}$$

Finally, the confusion matrix, defined in Table VIII, provides a comprehensive view of the model's performance. Each row of the matrix represents the instances in an actual class, while each column represents the instances in a predicted class.

D. Performance Evaluation

The performance evaluation results are shown in Fig. 4–Fig. 7, with multiclass confusion matrices presented in Fig. 8–Fig. 11.

Across both DS, the BERT-based BM demonstrates high accuracy, particularly in classifying BENIGN traffic and common attack types like DDoS and Port Scan. However, it shows variability in detecting more complex attacks, such as Web-based attacks and infiltration on the CIC-IDS 2017 dataset,

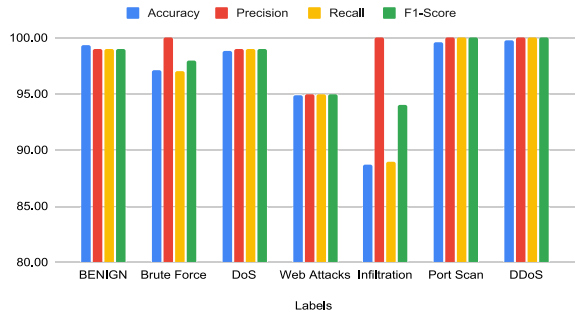


Fig. 5. Performance evaluation of BERT-large-super model on the CIC-IDS 2017 dataset.

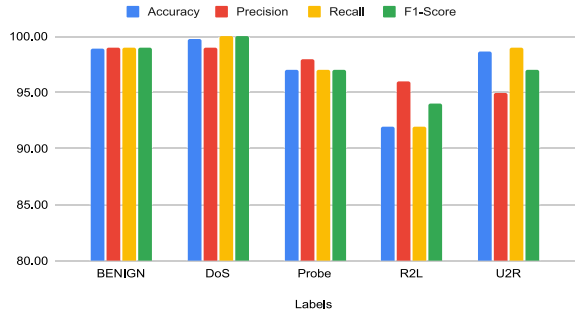


Fig. 6. Performance evaluation of BERT-base super model on the NSL-KDD 2009 dataset.

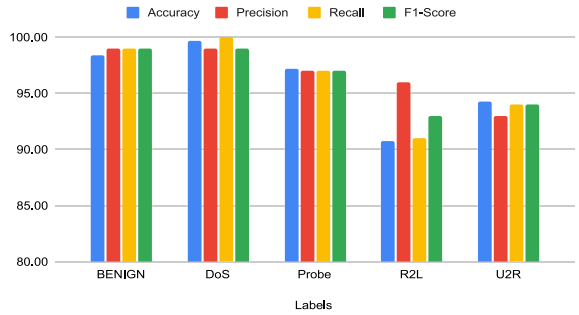


Fig. 7. Performance evaluation of BERT-large-super model on the NSL-KDD 2009 dataset.

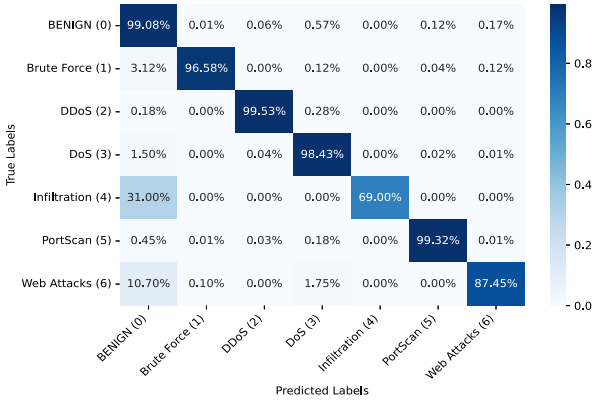


Fig. 8. Confusion matrix of multiclass classification using BERT-base *supermodel* on CIC-IDS 2017 dataset.

and faces challenges with the R2L and U2R categories on the NSL-KDD 2009 dataset. Despite high precision, indicating few false positives, BERT-Base struggles with false negatives, especially for less frequent attack types.

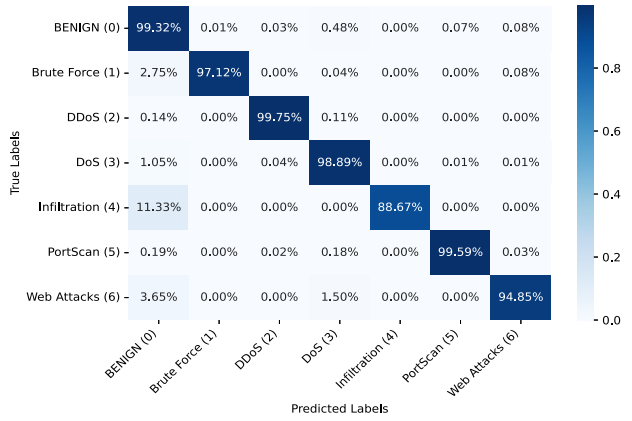


Fig. 9. Confusion matrix of multiclass classification using BERT-large-*supermodel* on CIC-IDS 2017 dataset.

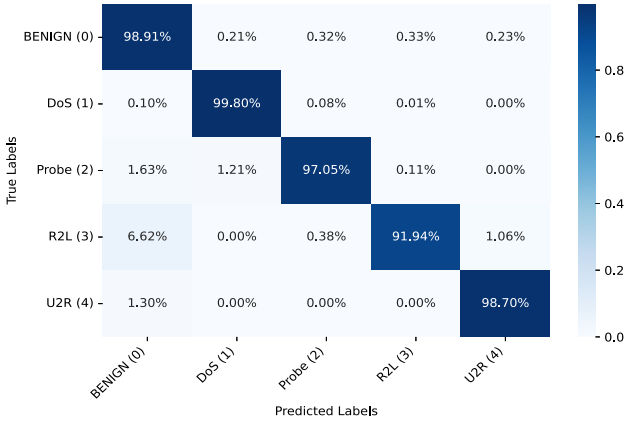


Fig. 10. Confusion matrix of multiclass classification using BERT-base *supermodel* on NSL-KDD 2009 dataset.

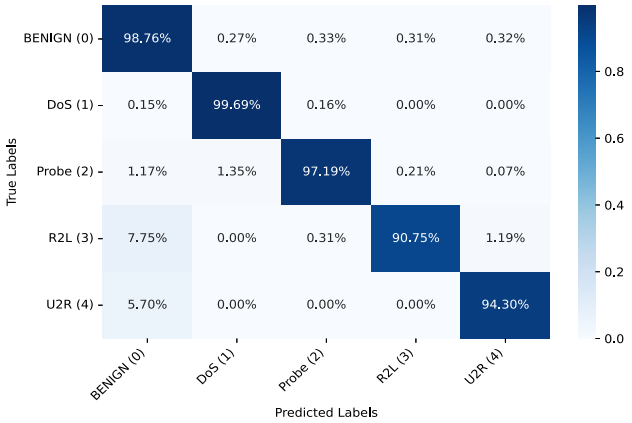


Fig. 11. Confusion matrix of multiclass classification using BERT-large-*supermodel* on NSL-KDD 2009 dataset.

On the other hand, the BERT-Large model consistently outperforms BERT-Base on the CIC-IDS 2017 dataset, achieving superior results in accuracy, precision, recall, and F1-Score across most categories, particularly for Web attacks and infiltration. However, when applied to the NSL-KDD 2009 dataset, BERT-Base performs slightly better overall, particularly in terms of accuracy and precision, due to its better balance between model complexity and the dataset's characteristics.

Our experimental results indicate that BERT-Large-based *supermodel* performs better on the higher number of features, as seen with CIC-IDS 2017, which contains 79 features, while BERT-Base excels on DS with fewer features, such as NSL-KDD 2009, which has 42 features. This suggests that the increased model capacity of BERT-Large is beneficial for capturing complex feature interactions in high-dimensional data, whereas BERT-Base *supermodel* remains effective in scenarios with lower feature complexity, offering a more efficient alternative for resource-constrained environments.

To evaluate the effectiveness of our proposed BERT and MLP model, Table IX compares its accuracy, precision, recall, and F1-score against several state-of-the-art classifiers on the same CIC-IDS 2017 and NSL-KDD 2009 DS. For the CIC-IDS 2017 dataset, our model achieved an accuracy of 99.39%, which is higher than the other models compared. The closest competitor, a CNN-based approach [38], achieved an accuracy of 98.79%. Additionally, our model excels in terms of precision, recall, and F1-score, all of which are also at 99.39%, surpassing other methods, such as the CNN-LSTM [39] hybrid model that reaches 98.56% accuracy and 99.25% F1-score. On the NSL-KDD 2009 dataset, our model also shows superior performance, with an accuracy of 99.33%, compared to other methods like the Ensemble Learning-DNN [40], which achieved 98.90%. Our model maintains a balanced accuracy, recall, and F1-score of 99.33%.

To address potential concerns regarding sequential dependency between edge nodes, we clarify that the proposed TL framework does not enforce strict sequential execution. While EN-B benefits from the pretrained model generated at EN-A, it is capable of performing standalone training and intrusion detection using its own dataset. The performance evaluation of the proposed TL-based approach, leveraging both BERT-Base and BERT-Large models at individual edge nodes, is detailed in Table X. The standalone BERT-Base and BERT-Large models consistently demonstrate high accuracy across both nodes, achieving 99.37% and 98.94% accuracy at EN-A, and 99.33% and 99.35% at EN-B, respectively. The *supermodel* BERT-Large, designed to handle data from both nodes, achieves competitive results with 99.23% accuracy at Node A, 98.38% at Node B, and an aggregated accuracy of 99.33% when tested across EN-A and EN-B. These results underscore the robustness of the proposed *supermodel*, which maintains generalization, high precision, recall, and F1-score, even in heterogeneous data distributions without limiting the autonomy of individual nodes.

E. Real-World Applicability of Proposed Approach

The model is designed for distributed edge environments, where intrusion detection must occur in real-time with minimal computational overhead. By leveraging TL, the model adapts to different network conditions without requiring extensive retraining for each new dataset or device. The unified *supermodel* is only 3.10 MB in size, making it highly efficient for deployment on IoT devices with limited storage and processing capabilities. Traditional DL-based IDS models are often too computationally expensive for IoT devices, whereas

TABLE IX
PERFORMANCE EVALUATION OF TL-BASED *Supermodel* WITH STATE-OF-THE-ART CLASSIFIERS USING CIC-IDS 2017 AND NSL-KDD 2009 DS

Dataset	Study	Classifier	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
CIC-IDS 2017	Proposed	BERT and MLP	**99.39**	**99.39**	**99.39**	**99.99**
	[38]	Convolutional Neural Network (CNN)	98.79	98.80	98.77	98.77
	[41]	Self-Organizing Map (SOM)	80.00	77.40	81.80	80.00
	[39]	CNN-Long Short Term Memory (LSTM)	98.56	98.50	99.00	99.25
	[39]	CNN-Recurrent Neural Networks (RNN)	97.24	97.50	97.25	96.50
	[39]	CNN-Gated Recurrent Unit (GRU)	96.12	96.25	96.25	95.50
NSL-KDD 2009	Proposed	BERT and MLP	**99.33**	**99.34**	**99.33**	**99.33**
	[42]	Decision Tree	97.40	97.40	97.10	97.20
	[41]	SOM	91.00	91.00	91.40	91.00
	[43]	Autoencoder	79.74	82.22	79.74	76.47
	[40]	Ensemble Learning-Deep Neural Network (DNN)	98.90	99.90	98.71	99.31
	[44]	Non-Symmetric Deep Autoencoder (NDAE)	85.42	100.00	85.42	87.37

TABLE X
PROPOSED APPROACH PERFORMANCE EVALUATION AT INDIVIDUAL NODES

Individual Nodes (A and B)	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Standalone BERT-Base at Node A	99.37	99.37	99.37	99.37
Standalone BERT-Base at Node B	99.33	99.34	99.33	99.33
Standalone BERT-Large at Node A	98.94	98.96	98.94	98.94
Standalone BERT-Large at Node B	99.35	99.34	99.35	99.34
Supermodel BERT-Large at Node A	99.23	99.23	99.23	99.23
Supermodel BERT-Large at Node B	98.38	98.38	98.38	98.38
Supermodel BERT-Large at Node A & B	99.33	99.34	99.33	99.33

our BERT-MLP-based approach optimizes performance while maintaining lightweight deployment. The performance evaluation (Table IX) demonstrates that the proposed approach outperforms multiple SOTA IDS solutions in terms of accuracy, precision, recall, and F1-score across two benchmark DS. These results validate the model's robustness across diverse network traffic scenarios, making it suitable for real-world cybersecurity applications in IoT networks, smart cities, industrial control systems, and autonomous vehicles. The TL mechanism enables the model to continuously learn from new edge nodes, allowing it to adapt to previously unseen attack patterns without requiring complete retraining. This adaptability is crucial for zero-day attack detection, ensuring that the IDS remains effective in evolving network environments.

However, deploying the proposed TL-enabled IDS in real-world edge environments presents several challenges, including hardware constraints, network latency, security vulnerabilities, and adaptability to evolving threats. IoT devices often have limited computational power and storage, necessitating lightweight models; our optimized 3.10 MB *supermodel* facilitates efficient deployment, but further compression techniques, such as quantization and pruning, can enhance feasibility. Additionally, as the *supermodel* is periodically shared with end devices, network bandwidth limitations and latency issues must be addressed through asynchronous updates and differential synchronization techniques to ensure real-time adaptability. Furthermore, given the highly dynamic nature of cyber threats, continuous model updating is essential, and while TL allows adaptability across DS, integrating FL and online incremental learning can further enhance real-time detection capabilities.

There also security concerns, such as poisoning attacks, adversarial perturbations, and model inversion risks, that require robust countermeasures, including adversarial training, secure aggregation in federated settings, and blockchain-based trust mechanisms to maintain IDS reliability.

Adversarial attacks involve manipulating input data to either evade detection or degrade model performance. The two primary types are evasion attacks, where adversaries craft adversarial samples to bypass detection, and poisoning attacks, where malicious data is injected into the training set to compromise model accuracy.

To enhance the robustness of our TL-enabled IDS, several countermeasures can be employed, such as adversarial training, defensive distillation, feature squeezing and ensemble learning. While our current work does not explicitly focus on adversarial robustness, these countermeasures provide a foundation for enhancing the security of TL-enabled IDS in edge deployments and will be addressed in future work.

One of the primary limitations encountered during this research was the computational intensity associated with transformer-based models, such as BERT, and DL techniques, like the MLP. While essential for robust performance, this demand for substantial GPU resources may hinder scalability and real-time applicability, especially in environments or organizations with limited access to high-performance computing infrastructure. Future work should explore optimization strategies, distributed computing, or hardware acceleration to overcome this constraint, specifically focusing on integrating edge computing resources. Leveraging edge computing capabilities can distribute the computational load to local devices, enhancing scalability and real-time responsiveness in resource-constrained settings. Acknowledging these computational challenges is crucial for transparency and a comprehensive understanding of the proposed approach's practical implications, ensuring the security and stability of our interconnected world in the face of digital evolution.

VI. Conclusion

Our research presents a groundbreaking approach to fortifying IDS by seamlessly integrating TL into edge computing. Our methodology harnesses the power of BERT for semantic feature extraction, utilizing both BERT-Base and BERT-Large architectures. To address the class imbalance, we applied SMOTE. The enriched feature data is utilized to train an MLP using a TL collaborative strategy, resulting in the development of a joint *supermodel* demonstrating exceptional adaptability and proficiency across various intrusion scenarios mapped by heterogeneous DS collected from different edge nodes. In real-world scenarios, the model's accuracy could be challenged by sudden shifts in network traffic patterns or the introduction of novel intrusion techniques that deviate significantly from those observed during training. For example, a DDoS attack with previously unseen traffic characteristics or a sophisticated phishing attempt involving encrypted traffic may introduce anomalies that the model has not encountered, potentially resulting in misclassification. Similarly, unexpected spikes in benign traffic or the emergence of new attack vectors could also reduce the model's effectiveness. To mitigate these challenges and ensure consistent performance, we propose incorporating threat intelligence feeds that provide real-time updates on emerging threats. By integrating this information, the model can stay informed of new attack patterns and adjust accordingly. Furthermore, training the model on data from diverse sources and environments, rather than relying solely on data from local edge nodes, can enhance its ability to generalize. This approach will make the model more resilient to variations in traffic patterns across different network environments, as shown by our validation experiments on two widely used DS.

References

[1] B. Panchali, "Edge computing-background and overview," in *Proc. Int. Conf. Smart Syst. Inventive Technol. (ICSSIT)*, 2018, pp. 580–582.

[2] G. Kaur and R. S. Batth, "Edge computing: Classification, applications, and challenges," in *Proc. 2nd Int. Conf. Intell. Eng. Manag. (ICIEM)*, 2021, pp. 254–259.

[3] P. Dini, A. Elhanashi, A. Begni, S. Saponara, Q. Zheng, and K. Gasmi, "Overview on intrusion detection systems design exploiting machine learning for networking cybersecurity," *Appl. Sci.*, vol. 13, no. 13, p. 7507, Jun. 2023.

[4] S. Hafeez and N. Kathirisetty, "Effects and Comparison of different Data pre-processing techniques and ML and deep learning models for sentiment analysis: SVM, KNN, PCA with SVM and CNN," in *Proc. 1st Int. Conf. Artif. Intell. Trends Pattern Recognit. (ICAITPR)*, 2022, pp. 1–6.

[5] G. Zhu, H. Zhao, H. Liu, and H. Sun, "A novel LSTM-GAN algorithm for time series anomaly detection," in *Proc. Prognostics Syst. Health Manag. Conf. (PHM-Qingdao)*, 2019, pp. 1–6.

[6] B. Aslahi-Shahri et al., "A hybrid method consisting of GA and SVM for intrusion detection system," *Neural Comput. Appl.*, vol. 27, pp. 1669–1676, Aug. 2016.

[7] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security Def. Appl.*, 2009, pp. 1–6.

[8] Z. Ali, W. Tiberti, A. Marotta, and D. Cassioli, "Empowering network security: Bert transformer learning approach and MLP for intrusion detection in imbalanced network traffic," *IEEE Access*, vol. 12, pp. 137618–137633, 2024.

[9] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.

[10] M. Mimura and H. Tanaka, "Reading network packets as a natural language for intrusion detection," in *Proc. 20th Int. Conf. Inf. Security Cryptol. (ICISC)*, 2018, pp. 339–350.

[11] S. Latif, W. Boulila, A. Koubaa, Z. Zou, and J. Ahmad, "DTL-IDS: An optimized intrusion detection framework using deep transfer learning and genetic algorithm," *J. Netw. Comput. Appl.*, vol. 221, Jan. 2024, Art. no. 103784.

[12] H. Zhang, L. Huang, C. Q. Wu, and Z. Li, "An effective convolutional neural network based on smote and gaussian mixture model for intrusion detection in imbalanced dataset," *Comput. Netw.*, vol. 177, Aug. 2020, Art. no. 107315.

[13] B. Zong et al., "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–19.

[14] K. Alrawashdeh and C. Purdy, "Toward an online anomaly intrusion detection system based on deep learning," in *Proc. 15th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2016, pp. 195–200.

[15] J. Zhang, C. Luo, M. Carpenter, and G. Min, "Federated learning for distributed IIoT intrusion detection using transfer approaches," *IEEE Trans. Ind. Informat.*, vol. 19, no. 7, pp. 8159–8169, Jul. 2023.

[16] J. H. Joloudari, A. Marefat, M. A. Nematollahi, S. S. Oyelere, and S. Hussain, "Effective class-imbalance learning based on smote and convolutional neural networks," *Appl. Sci.*, vol. 13, no. 6, p. 4006, 2023.

[17] Q. Yang, "An introduction to transfer learning," in *Proc. Adv. Data Min. Appl.*, 2008, p. 1.

[18] N. Alhussien, A. Aleroud, R. Rahaeimehr, and A. Schwarzmann, "Triggerability of backdoor attacks in multi-source transfer learning-based intrusion detection," in *Proc. IEEE/ACM Int. Conf. Big Data Comput., Appl. Technol. (BDCAT)*, 2022, pp. 40–47.

[19] Y. Cheng, J. Lu, D. Niyato, B. Lyu, J. Kang, and S. Zhu, "Federated transfer learning with client selection for intrusion detection in mobile edge computing," *IEEE Commun. Lett.*, vol. 26, no. 3, pp. 552–556, Mar. 2022.

[20] W. Yutao, L. Zhongtian, B. Yi, L. Jie, X. Fangzheng, and B. Yu, "Internet of Things intrusion detection system based on transfer learning," in *Proc. IEEE 2nd Int. Conf. Electron. Technol., Commun. Inf. (ICETCI)*, 2022, pp. 25–30.

[21] S. T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, and R. Islam, "Dependable intrusion detection system for IoT: A deep transfer learning based approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1006–1017, Jan. 2023.

[22] Y. K. Saheed, O. H. Abdulganiyu, and T. A. Tchakoucht, "Modified genetic algorithm and fine-tuned long short-term memory network for intrusion detection in the Internet of Things networks with edge capabilities," *Appl. Soft Comput.*, vol. 155, Apr. 2024, Art. no. 111434.

[23] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.

[24] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.

[25] Y. Meidan et al., "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul.–Sep. 2018.

[26] H. S. M. Masum, "A transfer learning with deep neural network approach for network intrusion detection," *Int. J. Intelll. Comput. Res.*, vol. 12, no. 1, pp. 1–9, 2021.

[27] A. Bagaskara and M. Suryanegara, "Evaluation of VGG-16 and VGG-19 deep learning architecture for classifying dementia people," in *Proc. 4th Int. Conf. Comput. Informat. Eng. (IC2IE)*, 2021, pp. 1–4.

[28] D. Sinha and M. El-Sharkawy, "Thin MobileNet: An enhanced MobileNet architecture," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, 2019, pp. 280–285.

[29] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[30] Y. K. Saheed and S. Misra, "CPS-IoT-PPDNN: A new explainable privacy preserving DNN for resilient anomaly detection in cyber-physical systems-enabled IoT networks," *Chaos, Solitons & Fractals*, vol. 191, Feb. 2025, Art. no. 115939.

[31] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[32] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "X-IIoTID: A connectivity-agnostic and device-agnostic intrusion data set for industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3962–3977, Mar. 2022.
[33] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.
[34] A. Kumar, S. Kumar, V. Kumar, A. Kumari, A. Saini, and S. Gupta, "Edge computing based IDS detecting threats using machine learning and PyCaret," in *Proc. Int. Conf. Comput. Intell. Sustain. Eng. Solut. (CISES)*, 2023, pp. 668–673.
[35] P. Mahadevappa, R. K. Murugesan, R. Al-amri, R. Thabit, A. H. Al-Ghushami, and G. Alkawsi, "A secure edge computing model using machine learning and IDS to detect and isolate intruders," *MethodsX*, vol. 12, Jun. 2024, Art. no. 102597.
[36] H. Bangui and B. Buhnova, "Lightweight intrusion detection for edge computing networks using deep forest and bio-inspired algorithms," *Comput. Elect. Eng.*, vol. 100, May 2022 Art. no. 107901.
[37] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116.
[38] M. A. Siddiqi and W. Pak, "Tier-based optimization for synthesized network intrusion detection system," *IEEE Access*, vol. 10, pp. 108530–108544, 2022.
[39] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic," *Digit. Commun. Netw.*, vol. 10, pp. 190–204, Feb. 2024.
[40] A. Thakkar and R. Lohiya, "Attack classification of imbalanced intrusion data for IoT network using ensemble-learning-based deep neural network," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11888–11895, Jul. 2023.
[41] J. Ables et al., "Creating an explainable intrusion detection system using self organizing maps," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2022, pp. 404–412.
[42] R.-F. Hong, S.-C. Horng, and S.-S. Lin, "Machine learning in cyber security analytics using NSL-KDD dataset," in *Proc. Int. Conf. Technol. Appl. Artif. Intell. (TAAI)*, 2021, pp. 260–265.
[43] C. Zhang, F. Ruan, L. Yin, X. Chen, L. Zhai, and F. Liu, "A deep learning approach for network intrusion detection based on NSL-KDD dataset," in *Proc. IEEE 13th Int. Conf. Anti-Counterfeiting, Security, Identif. (ASID)*, 2019, pp. 41–45.
[44] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 41–50, Feb. 2018.



Zeeshan Ali (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in computer science from the National College of Business Administration and Economics, Lahore, Pakistan, in 2021 and 2022, respectively. He is currently pursuing the Ph.D. degree with the Department of Information Engineering, Computer Science, and Mathematics, University of L'Aquila, L'Aquila, Italy, supervised by Dr. D. Cassioli.

He is a Cybersecurity Researcher with the Department of Information Engineering, Computer Science, and Mathematics, University of L'Aquila. He has hands-on experience with state-of-the-art cybersecurity tools and advanced technologies, including large-language models and generative AI. He previously served as the National Startup Founder with the Punjab Information Technology Board, Lahore. His research interests encompass cybersecurity, artificial intelligence, machine learning, the Internet of Things, and software quality assurance.



Walter Tiberti (Member, IEEE) received the B.Sc. and M.Sc. degrees in computer engineering and the Ph.D. degree in information and communication technology from the University of L'Aquila, L'Aquila, Italy, in 2013, 2016, and 2020, respectively, where his research focused on embedded systems security, particularly the security of wireless sensor networks.

He is currently an Assistant Professor with the Department of Information Engineering, Computer Science, and Mathematics, University of LŠAquila. He has also been a Visiting Researcher with the Research Center in Real-Time and Embedded Computing Systems, Porto, Portugal. His research interests include software security, network security, and cryptography.

Dr. Tiberti serves as a technical committee member for various IEEE conferences and is actively involved in national initiatives aimed at advancing cybersecurity education and training in Italy.



Andrea Marotta (Member, IEEE) received the M.Sc. degree in computer engineering and the Ph.D. degree in information and communications technology from the University of L'Aquila, L'Aquila, Italy, in 2015 and 2019, respectively.

He is Assistant Professor (Tenure) with the Department of Information Engineering, Computer Science and Mathematics, University of L'Aquila. He was Visiting Researcher with the Group for Research on Wireless, Instituto Superior Técnico/University, Lisbon, Portugal, and the Research Institute of Communication, Information and Perception Technologies, Scuola superiore di studi universitari e di perfezionamento Sant'Anna, Pisa, Italy, and the Computer Networks Laboratory, University of California at Davis, Davis, CA, USA. He performs researches on optical and mobile networks, network reliability, and multiaccess-edge computing.

Dr. Marotta actively participates in the TPC of IEEE Communications Society conferences.



Dajana Cassioli (Senior Member, IEEE) is an Associate Professor of Telecommunications Engineering with the University of L'Aquila, L'Aquila, Italy, where she served as the Elected Head for the Study Program in "Telecommunications Engineering: Advanced Technologies and Services" from 2021 to 2024. She co-authored more than 100 articles published in the most renowned international journals, magazines, and conference proceedings. She was a Summer Manager with the Wireless Systems Research Department, AT&T Laboratories-Research, NJ, USA, in 2000. She was a Short-Term Visiting Scholar with the University of Southern California, Los Angeles, CA, USA, hosted by Prof. Andy Molisch, in 2022. Her main research interests are in wireless communications, 5G/B5G networks and cybersecurity.

Dr. Cassioli is the Chair of the IEEE ComSoc RCC SiG on Propagation Channels for 5G&B, the Chair of the SiG on Distributed and Massive MIMO of the IEEE P1944 Standard for Channel Models of Wireless Systems and the Diversity, Equity and Inclusion Activity Coordinator of the IEEE Italy Section. She is the Past Chair of the IEEE WIE AG Italy Section from 2016 to 2022 and the IEEE VT06/COM19 Italy Chapter from 2011 to 2017. Since 2015, she is the Coordinator of the University of L'Aquila Node of the CINI National Laboratory of Cybersecurity, where she led the CyberEquality WG from 2020 to 2021. She has been awarded the ERC StG VISION (Video-oriented UWB-based Intelligent Ubiquitous Sensing) in 2010 and the ERC PoC Grant iCARE (MobIle health-Care system for monitoring toxicity and symptoms in Cancer patients Receiving Disease-Oriented Therapy) in 2016. She was the CEO in 2014–2018, and 2019 of the spin-off of the University of L'Aquila "Smartly: Natives of Smart Living srl," which designs and markets advanced ICT solutions to improve the quality of life. She served as the Co-Chair for Globecom 2025 WC, ICC 2024 MWN, ICC 2023 CISS, PIMRC2018 Industry, RTSI WIE Chair in 2018, 2019, and 2020, MELECON2020 and MetroInd4.0, and the TPC member of several International Conferences, including ICC, PIMRC, VTC, and GLOBECOM. She participated in the definition of the standard channel model for the IEEE 802.15.4 standard in 2005. She is an Associate Editor of *IET Electronic Letters*, and an Executive Editor of WILEY INTERNET TECHNOLOGY LETTERS and *Transactions on Emerging Telecommunications Technology* and served as an Associate Editor for IEEE COMMUNICATIONS LETTERS from 2018 to 2022.